

Guia practica: Docker para microservicios Spring Boot

Imagen, contenedor, Flyway, MySQL, puertos, Eureka y API Gateway

1. Que hicimos en esta practica

En esta prueba tomamos un microservicio Spring Boot llamado **pedidos**, creamos una imagen Docker y luego ejecutamos esa imagen como un contenedor. Al principio el servicio no arrancaba porque Flyway intentaba conectarse a MySQL y no podia llegar correctamente a la base de datos. Despues corregimos las variables de conexion, especialmente la URL de MySQL y la contrasena vacia, y finalmente el microservicio quedo corriendo en Docker y respondiendo desde Postman con codigo **200 OK**.

Flujo general:

Paso	Que significa
1. docker build	Construye una imagen Docker del proyecto Spring Boot.
2. docker images	Permite verificar que la imagen existe, por ejemplo pedidos:latest.
3. docker run	Crea y ejecuta un contenedor a partir de esa imagen.
4. Variables -e	Sobrescriben propiedades de Spring Boot, como URL, usuario y password de MySQL.
5. Postman	Permite probar que el microservicio responde desde <code>http://localhost:8083/api/v1/pedidos</code> .

2. Diferencia entre imagen y contenedor

Esta es la duda mas importante. Una **imagen** y un **contenedor** no son lo mismo.

Concepto	Explicacion simple	Ejemplo en tu practica
Imagen	Es una plantilla o paquete listo para ejecutar. Contiene el .jar, Java y todo lo necesario para levantar el microservicio.	pedidos:latest
Contenedor	Es una instancia en ejecucion creada a partir de una imagen. Es el microservicio ya corriendo.	contenedor llamado pedidos
Relacion	De una misma imagen puedes crear varios contenedores. La imagen queda guardada aunque borres el contenedor.	docker rm pedidos borra el contenedor, no la imagen.

Ejemplo con una analogia:

Imagen: es como el plano o molde de una casa. **Contenedor:** es la casa construida y funcionando. Puedes borrar una casa, pero el plano sigue existiendo para construir otra igual.

3. Comandos que usamos y para que sirven

Ver las imagenes disponibles:

```
docker images
```

En tu caso aparecio una imagen llamada **pedidos** con tag **latest**. Eso significa que la imagen ya estaba creada.

Eliminar un contenedor fallido:

```
docker rm pedidos
```

Esto elimina el contenedor, pero no elimina la imagen. Solo tendrías que reconstruir la imagen si cambiaste código o si ejecutaste **docker rmi pedidos**.

Ejecutar el microservicio con MySQL en tu PC y password vacia:

```
docker run --name pedidos -p 8083:8083 \
-e SPRING_DATASOURCE_URL="jdbc:mysql://host.docker.internal:3306/pedidos_db" \
-e SPRING_DATASOURCE_USERNAME=root \
-e SPRING_DATASOURCE_PASSWORD="" \
pedidos:latest
```

4. Por que fallo primero

El primer error importante fue **Communications link failure**. Eso significa que la aplicacion dentro del contenedor no podia comunicarse con MySQL. Esto pasa mucho cuando en Spring Boot se usa **localhost** en la URL de la base de datos.

Dentro de Docker, localhost significa el propio contenedor, no tu computador. Por eso usamos host.docker.internal, que permite que el contenedor se conecte al MySQL que esta corriendo en Windows.

El segundo error fue **Access denied for user root using password YES**. Eso paso porque se ejecuto el contenedor con password **root**, pero tu MySQL no tenia contrasena. La solucion fue ejecutar con:

```
-e SPRING_DATASOURCE_PASSWORD=""
```

Cuando esa variable queda vacia, Spring Boot intenta conectarse con usuario root y sin contrasena, que era la configuracion real de tu proyecto de practica.

5. Que tiene que ver Flyway

Flyway es la herramienta que ejecuta migraciones SQL al iniciar la aplicacion. Antes de que el microservicio termine de arrancar, Flyway intenta conectarse a la base de datos para crear o actualizar tablas.

Por eso parecia que Flyway era el problema, pero en realidad Flyway solo estaba mostrando el problema de conexion a MySQL. Si MySQL no esta prendido, la base no existe o la password esta mala, Flyway falla y Spring Boot no arranca.

Regla practica: Flyway no suele crear la base de datos completa. Debes tener creada la base, por ejemplo pedidos_db. Luego Flyway crea o actualiza las tablas con los scripts de migracion.

6. Puertos: contenedor, PC y Postman

El comando:

```
docker run --name pedidos -p 8083:8083 pedidos:latest
```

significa que el puerto **8083 de tu computador** se conecta con el puerto **8083 del contenedor**. Por eso pudiste probar en Postman:

```
http://localhost:8083/api/v1/pedidos
```

7. Si mi proyecto tiene Eureka y API Gateway, sirve igual con puerto 8080?

Respuesta corta: **si, pero depende de que estes ejecutando.**

En una arquitectura con API Gateway, lo normal es que el cliente, Postman o el navegador entren por el **Gateway**, por ejemplo:

```
http://localhost:8080/api/v1/clientes
http://localhost:8080/api/v1/equipos
http://localhost:8080/api/v1/ot
```

Pero eso no significa que todos los microservicios corran realmente en el puerto 8080. El Gateway corre en 8080 y redirige las solicitudes hacia los microservicios internos, que pueden estar en 8082, 8083, 8084, etc.

Ejemplo local:

Servicio	Puerto tipico local	Funcion
API Gateway	8080	Punto unico de entrada para Postman o navegador.
Eureka Server	8761	Registro donde los microservicios se inscriben.
Cliente MS	8082	Gestiona clientes.
Equipo MS	8083	Gestiona equipos.
Orden Trabajo MS	8084	Gestiona ordenes de trabajo.

8. Docker con varios microservicios

Si corres solo un microservicio con Docker, puedes usar **docker run**. Pero si tienes varios microservicios, Eureka, Gateway y bases de datos, lo mas ordenado es usar **docker-compose.yml**.

Con Docker Compose puedes levantar varios contenedores juntos: gateway, eureka, cliente, equipo, orden-trabajo y bases de datos. Todos quedan en una misma red interna y se pueden comunicar por nombre de servicio.

Ejemplo conceptual:

```
gateway -> eureka
cliente-ms -> eureka
equipo-ms -> eureka
ot-ms -> eureka
gateway -> enruta hacia cliente-ms, equipo-ms y ot-ms
```

Importante: dentro de Docker Compose, un microservicio no debería llamar a otro usando localhost. Debe usar el nombre del servicio o Eureka. Por ejemplo, la base de datos podría llamarse mysql-cliente, y la URL JDBC seria jdbc:mysql://mysql-cliente:3306/cliente_db.

9. Que responder en defensa

Si te preguntan que hiciste con Docker, puedes responder:

"Cree una imagen Docker del microservicio Spring Boot y luego ejecute un contenedor a partir de esa imagen. Configure el mapeo de puertos con -p 8083:8083 y pase las variables de entorno de la base de datos con -e, porque el contenedor necesitaba conectarse al MySQL local. El problema inicial fue que dentro de Docker localhost no apunta a mi computador, por eso use host.docker.internal. Además, como la base de datos no tenía contraseña, configure SPRING_DATASOURCE_PASSWORD como cadena vacía. Finalmente el contenedor quedo en estado Running y probe el endpoint en Postman con respuesta 200 OK."

Si te preguntan por el Gateway y los puertos:

"Cuando uso API Gateway, el cliente consume el sistema por el puerto del Gateway, normalmente 8080. Los microservicios mantienen sus propios puertos internos o se registran en Eureka. El Gateway centraliza la entrada y redirige las solicitudes hacia el servicio correspondiente. En Docker, el puerto que se expone al computador se define con -p, y en un escenario con varios servicios lo ideal es usar Docker Compose para que todos compartan una red interna."

10. Checklist antes de entregar o defender

- Docker Desktop encendido.
- Imagen creada: docker images.
- Contenedor corriendo: docker ps.
- Logs revisados: docker logs pedidos.
- Base de datos creada: pedidos_db.
- MySQL encendido en Laragon, XAMPP o servicio local.
- Endpoint probado en Postman con 200 OK.
- Saber explicar imagen vs contenedor.
- Saber explicar por que se usa host.docker.internal.
- Saber explicar que el Gateway usa 8080 como punto unico de entrada.

Resumen final:

Una imagen es el paquete construido. Un contenedor es la ejecucion de ese paquete. Docker no reemplaza Spring Boot ni MySQL: solo permite ejecutar la aplicacion en un ambiente aislado y reproducible. Para microservicios con Gateway y Eureka, el puerto 8080 normalmente corresponde al Gateway; los demas servicios pueden quedar en sus propios puertos o en una red interna de Docker.